HTML Quick-Styler

Naan Mudhalvan Project Report Submitted to **Manonmaniam Sundaranar University** in the Partial fulfilment of the Requirement for the award of the **Degree of Bachelor of Science in Computer Science.**

Submitted By

TEAM ID: SWTID-2026-3367

TEAM LEADER: S.KANAGA – 23082241802112018

TEAM MEMBER: M.KITHURU SULTHANA– 23082241802112019

TEAM MEMBER: M.MANI MEHALA– 23082241802112020

TEAM MEMBER: A.MARIDEVI– 23082241802112021

DEPARTMENT OF COMPUTER SCIENCE

WAVOO WAJEEHA WOMEN'S COLLEGE OF ARTS & SCIENCE

(Accredited by NAAC with 'B++' Grade)

KAYALPATNAM – 628 204

APRIL – 2025

CERTIFICATE

CERTIFICATE

This is to certify that the project work is entitled as **"HTML Quick-Styler" is a bonafide work by**

TEAM ID: SWTID-2026-3367

TEAM LEADER: S.KANAGA – 23082241802112018

TEAM MEMBER: M.KITHURU SULTHANA – 23082241802112019

TEAM MEMBER: M.MANI MEHALA – 23082241802112020

TEAM MEMBER: A.MARIDEVI – 23082241802112021

In partial fulfilment of the requirement for the award of the degree of **BACHELOR OF COMPUTER SCIENCE** from **MANONMANIAM SUNDARNAR UNIVERSITY, TIRUNELVELI,** during the academic year of **2025-2026.**

Signature of the Incharge Staff **Signature of the H.O.D**

EXTERNAL EXAMINERS

1.
2.

Place: Kayalpatnam

Date:

DECLARATION

DECLARATION

We hereby declare that the Naan Mudhalvan Project, entitled **" HTML Quick-Styler"** submitted for the **Bachelor of Science in Computer Science** at **Manonmaniam Sundaranar University** is done. This project was completed by the **Department of Computer Science** at Wavoo Wajeeha Women's College of Arts and Science in Kayalpatnam.

Signature of the Candidate

1. **S.KANAGA**
2. **M.KITHURU SULTHANA**
3. **M.MANI MEHALA**
4. **A.MARIDEVI**

DATE :

PLACE :Kayalpatnam

CONTENTS

S.NO	TABLE OF CONTENTS	PAGE NO
1	PROJECT DESCRPTION	8
2	SCENARIOS	10
3	A RCHITECTURE OVERVIEW	13
4	PROJECT WORKFLOW	16
5	PROJECT STRUCTURE	20
6	IMPLEMENTATION PHASE	22
7	CONCLUSION	22

PROJECT DESCRIPTION

Project Description

HTML Quick-Styler is an intelligent web page generation platform that leverages IBM's Granite 3.3 2B Instruct model to provide comprehensive HTML/CSS page creation with professional styling and responsive design. The platform addresses the challenge of rapid web development by delivering AI-powered HTML generation, automatic section detection, modern CSS styling, color palette management, and production-ready code output.

Using Granite 3.3 2B's advanced language capabilities, the system analyzes user descriptions (page title, section requirements, color preferences), generates semantic HTML structure, applies modern CSS styling with animations, creates responsive layouts, and provides multiple color palette options. The system ensures professional output through Gradio's user-friendly interface while maintaining high-quality design standards through sophisticated template generation and AI-powered content suggestions.

HTML Quick-Styler transforms web development into an intelligent, user-friendly experience through its modern Gradio interface, comprehensive feature set, and AI-powered analysis that provides personalized HTML pages considering design trends, accessibility standards, and responsive design principles.

SCENARIOS

Scenarios

Scenario 1: Portfolio Website Generation

A freelance designer wants to create a professional portfolio website quickly and efficiently. The system first detects the project requirements and analyzes the designer's style and design preferences. Based on this understanding, it generates a complete website structure that includes a visually appealing hero section with a welcoming message and a strong call-to-action to engage visitors immediately. It also builds a responsive portfolio gallery arranged in a clean grid layout to showcase projects effectively across devices. An about section is created to present the designer's professional bio in a compelling manner, helping visitors understand their background and expertise. To strengthen credibility, the system includes a testimonials section featuring client feedback. A contact form with proper input validation ensures smooth communication while preventing errors, and finally, a professional footer is added with social media links and essential information to complete the polished look of the website.

Scenario 2: SaaS Landing Page

A startup needs a powerful landing page for launching their new product. The system responds by creating a compelling hero banner that clearly communicates the product's value proposition and key messaging. It then generates a features showcase section, presenting product capabilities with visually appealing icons and concise descriptions to make the benefits easy to understand. A pricing comparison table is designed with clear plan options to help potential customers make informed decisions. To build trust and credibility, the system incorporates a testimonials section that highlights positive user experiences and social proof. Prominent and strategically placed call-to-action buttons are added throughout the page to guide visitors toward signing up or purchasing, ensuring strong conversion optimization.

Scenario 3: Corporate Website

A company requires a professional corporate website with multiple structured sections. The system delivers a clean and polished layout starting with a sticky navigation header that includes company branding and allows smooth navigation across sections. It creates a company overview section that clearly communicates the organization's mission, vision, and core values. A services grid is generated to present detailed descriptions of the company's offerings in an organized and visually structured manner. The website also includes a team members section with photos and short profiles to add a human touch and build trust with visitors. Additionally, a contact section is implemented with company information and an inquiry form for seamless

communication. The site is completed with a professional footer containing privacy policies and important links, ensuring a corporate-standard presentation.

Scenario 4: Custom Color Palette Exploration

A designer wants to experiment with different color schemes to find the perfect visual identity for their project. The system generates eight carefully curated pre-designed color palettes, including modern, vibrant, ocean, forest, sunset, luxury, creative, and minimal themes. Each palette comes with clearly defined hex color codes for every component, making implementation straightforward. The system provides a real-time preview feature, allowing the designer to instantly visualize how each color scheme affects the overall design. It ensures consistent color application across all page elements to maintain visual harmony. Additionally, dark theme–optimized backgrounds are included to support modern design trends and improve user experience in low-light viewing environments.

ARCHITECTURE OVERVIEW

Architecture Overview

HTML Quick-Styler is built as a modular platform combining Python backend with IBM Granite 3.3 2B Instruct model for intelligent HTML generation. The architecture prioritizes user experience, design quality, and code production through Gradio's interactive interface and sophisticated template generation.

Core Technologies

- Python 3.x: Core programming language for application logic
- IBM Granite 3.3 2B Instruct: AI model for intelligent content generation
- Gradio: Interactive web interface framework for user interaction
- HTML5/CSS3: Modern web standards for page generation
- JavaScript: Client-side interactivity and animations
- Google Colab: Cloud-based development environment
- Responsive Design: Mobile-first approach with media queries

Pre-requisites

Software Requirements

- Python 3.8+: Download from python.org
- Google Account: Required for Google Colab access
- Gradio: Install via pip (pip install gradio)
- Web Browser: Chrome, Firefox, Safari, or Edge
- Internet Connection: Required for Colab and model access

Knowledge Prerequisites

- Python Basics: Functions, classes, dictionaries, string manipulation
- HTML/CSS: Understanding of HTML structure and CSS styling
- Gradio Framework: Basic understanding of interface components
- Web Design: Color theory, responsive design principles
- Google Colab: Familiarity with notebook interface

Hardware Requirements

- Processor: Intel i3 or equivalent (Colab runs on cloud)
- RAM: Minimum 4GB local (Colab provides GPU/TPU)
- Storage: 1GB free space for generated files
- Display: Minimum 1280x720 resolution for interface

PROJECT WORKFLOW

Project Workflow

Phase 1: Environment Setup & Model Configuration

Establish Google Colab environment and configure Granite model

Activity 1.1: Set up Google Colab Environment

- Create new Google Colab notebook
- Enable GPU runtime for better performance
- Install required Python packages
- Verify environment configuration

Activity 1.2: Configure IBM Granite Model

- Access Hugging Face model repository
- Load Granite 3.3 2B Instruct model
- Test model inference capabilities
- Configure generation parameters

Activity 1.3: Install and Configure Gradio

- Install Gradio via pip
- Test basic Gradio interface
- Configure sharing options
- Verify interface rendering

Phase 2: Core Application Development

Build HTML generation engine and template system

Activity 2.1: Build Color Palette Generator

- Create ColorPaletteGenerator class
- Define 8 color palette schemes
- Implement palette selection logic
- Add color validation methods

Activity 2.2: Implement HTML Page Builder

- Create HTMLPageBuilder class

- Build section detection algorithm
- Implement template generation methods
- Create SEO meta tags generator

Activity 2.3: Develop CSS Generation Engine

- Build comprehensive CSS generator
- Implement responsive design rules
- Add animation and transition effects
- Create mobile-first media queries

Activity 2.4: Build Section Templates

- Create hero section template
- Build features grid template
- Implement portfolio gallery template
- Add contact form template
- Create pricing and testimonial templates

Phase 3: Gradio Interface Development

Create interactive user interface with multiple tabs

Activity 3.1: Interactive HTML Generation & Preview Interface

Activity 3 focuses on designing and implementing a **complete HTML generation and visualization interface** that enables users to create, customize, preview, and understand web page layouts through an intuitive, multi-section workflow.

Phase 4: Google Colab Deployment:

Integrate all components and perform comprehensive testing

Activity 4.1: End-to-End Integration

- Connect all interface components
- Test data flow between tabs
- Validate generation pipeline
- Test error handling

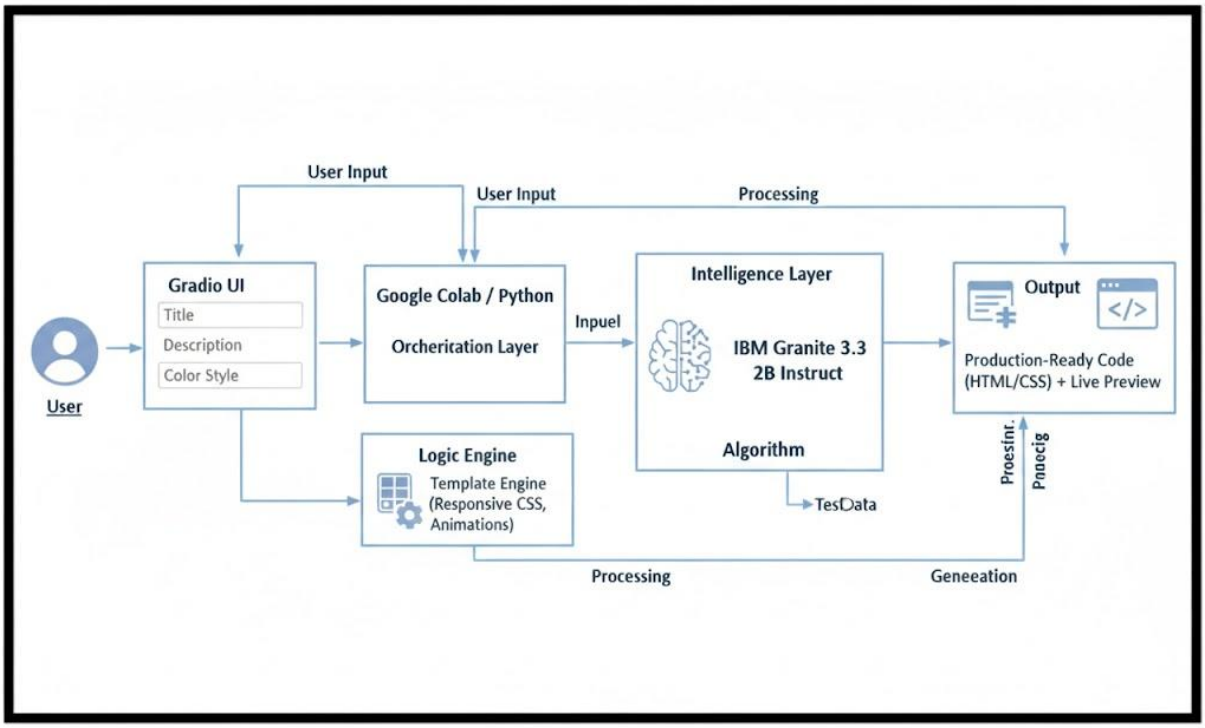
Phase 5: Deployment & Documentation

Deploy application and create comprehensive documentation

Activity 5.1: Functionality testing

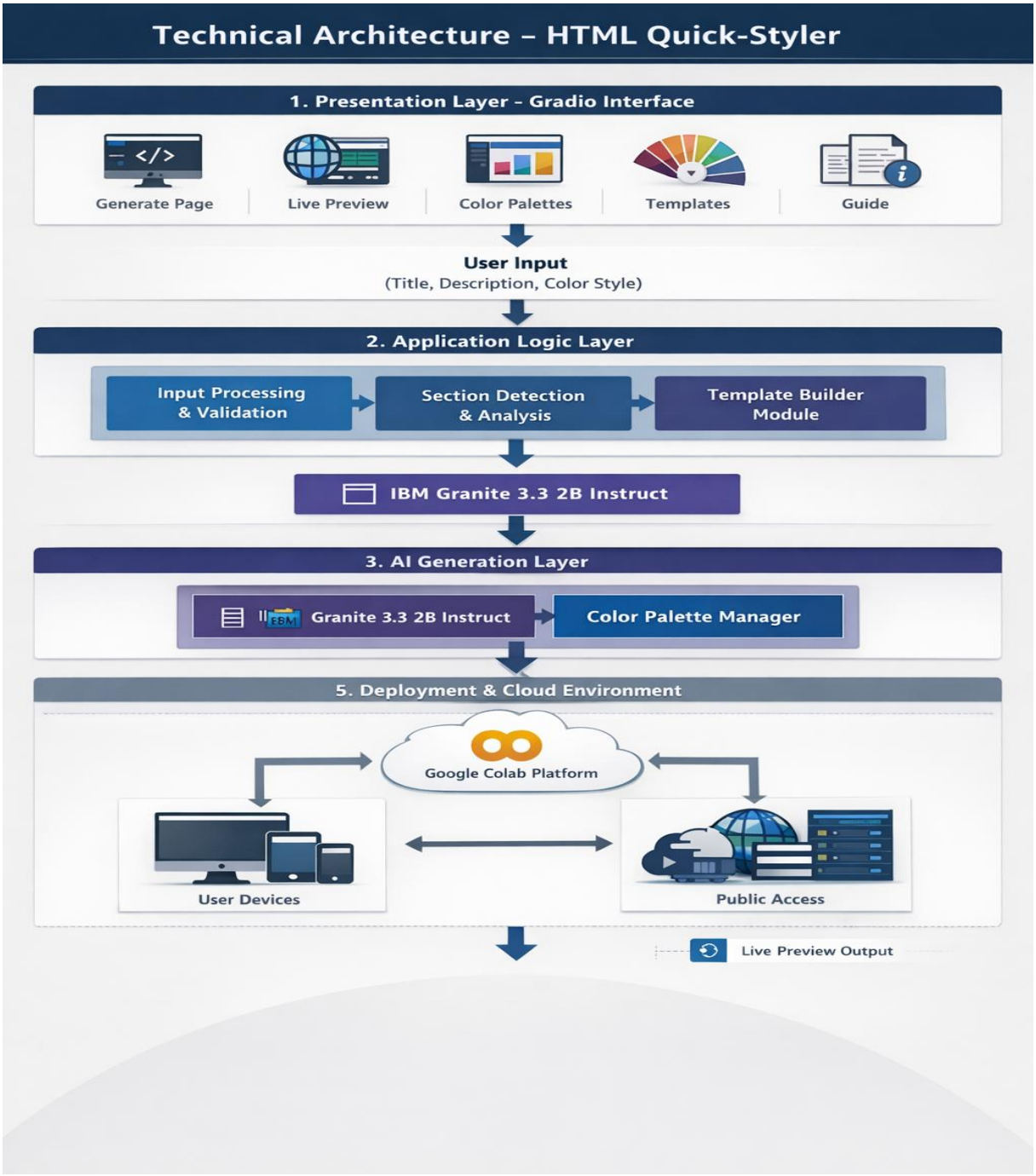
- Configure Gradio sharing settings
- Enable public access link
- Test external access
- Document deployment steps

Technical Architecture



PROJECT STRUCTURE

Project Structure:



IMPLEMENTATION PHASE

Implementation Phase

MILESTONE 1: Environment Setup & Model Configuration

Establish Google Colab environment and configure Granite model

Activity 1.1: Install Dependencies & Setup Environment

Create requirements.txt:

```
[ ]
!pip install -q gradio

import gradio as gr
import json
import re
from datetime import datetime
from typing import Tuple, Dict, List
import random
```

Activity 1.2: Configure IBM Granite Model

Access Hugging Face and load model:

Model: ibm-granite/granite-3.3-2b-instruct

```
# Load model directly
from transformers import AutoTokenizer, AutoModelForCausalLM

tokenizer = AutoTokenizer.from_pretrained("ibm-granite/granite-3.3-2b-instruct")
model = AutoModelForCausalLM.from_pretrained("ibm-granite/granite-3.3-2b-instruct")
messages = [
    {"role": "user", "content": "Who are you?"},
]
inputs = tokenizer.apply_chat_template(
  messages,
  add_generation_prompt=True,
  tokenize=True,
  return_dict=True,
  return_tensors="pt",
).to(model.device)

outputs = model.generate(**inputs, max_new_tokens=40)
print(tokenizer.decode(outputs[0][inputs["input_ids"].shape[-1]:]))

Loading weights: 100% 362/362 [00:04<00:00, 139.69it/s, Materializing param=model.norm.weight]
```

Colab Notebook:

MILESTONE 2: Core Application Development

Activity 2.1: Color Palette Generator Implementation

Create ColorPaletteGenerator class with 8 predefined color schemes:

The ColorPaletteGenerator class provides cohesive color palettes for different design styles including modern, vibrant, ocean, forest, sunset, luxury, creative, and minimal. Each palette includes primary, secondary, accent, background, surface, text primary, text secondary, and border colors optimized for dark theme interfaces.

```python
# HTML QUICK-STYLER - ADVANCED VERSION (FIXED)
# ==============================================================================

class ColorPaletteGenerator:
    """Generate visually cohesive color palettes"""

    @staticmethod
    def generate_palette(style: str = "modern") -> Dict[str, str]:
        """Generate a cohesive color palette based on style"""

        palettes = {
            'modern': {
                'primary': '#667eea',
                'secondary': '#764ba2',
                'accent': '#f093fb',
                'background': '#0f172a',
                'surface': '#1e293b',
                'text_primary': '#f1f5f9',
                'text_secondary': '#cbd5e1',
                'border': '#334155'
            },
            'vibrant': {
                'primary': '#ff6b6b',
                'secondary': '#ff8e72',
```

```
    },
    'ocean': {
        'primary': '#0099cc',
        'secondary': '#006699',
        'accent': '#00ccff',
        'background': '#0f172a',
        'surface': '#1e293b',
        'text_primary': '#f1f5f9',
        'text_secondary': '#cbd5e1',
        'border': '#334155'
    },
    'forest': {
        'primary': '#27ae60',
        'secondary': '#1e8449',
        'accent': '#52be80',
        'background': '#0f172a',
        'surface': '#1e293b',
        'text_primary': '#f1f5f9',
        'text_secondary': '#cbd5e1',
        'border': '#334155'
    },
    'sunset': {
        'primary': '#ff8c42',
        'secondary': '#ff6b35',
        'accent': '#ffa751',
        'background': '#0f172a',
        'surface': '#1e293b',
        'text_primary': '#f1f5f9',
        'text_secondary': '#cbd5e1',
        'border': '#334155'
    },
    'luxury': {
        'primary': '#1a1a1a',
        'secondary': '#333333',
        'accent': '#FFD700',
        'background': '#0f172a',
        'surface': '#1e293b',
        'text_primary': '#f1f5f9',
        'text_secondary': '#cbd5e1',
        'border': '#334155'
    },
```

```
        'creative': {
            'primary': '#9b59b6',
            'secondary': '#8e44ad',
            'accent': '#e74c3c',
            'background': '#0f172a',
            'surface': '#1e293b',
            'text_primary': '#f1f5f9',
            'text_secondary': '#cbd5e1',
            'border': '#334155'
        },
        'minimal': {
            'primary': '#2c3e50',
            'secondary': '#34495e',
            'accent': '#3498db',
            'background': '#0f172a',
            'surface': '#1e293b',
            'text_primary': '#f1f5f9',
            'text_secondary': '#cbd5e1',
            'border': '#334155'
        }
    }

    return palettes.get(style.lower(), palettes['modern'])
```

Activity 2.2: HTML Page Builder Implementation

The HTMLPageBuilder class includes:

- Section extraction from user descriptions
- Keyword detection for hero, features, about, portfolio, testimonials, pricing, contact, and footer sections
- SEO meta tags generation with Open Graph and Twitter Card support
- Individual section HTML generation with professional templates

```python
    @staticmethod
    def generate_meta_tags(title: str, description: str) -> str:
        """Generate SEO meta tags"""
        meta_tags = f'''    <meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<title>{title}</title>
<meta name="description" content="{description[:160]}">
<meta name="theme-color" content="#0f172a">
<meta property="og:title" content="{title}">
<meta property="og:description" content="{description[:160]}">
<meta property="og:type" content="website">
<meta name="twitter:card" content="summary_large_image">
<meta name="author" content="HTML Quick-Styler">'''
        return meta_tags

    @staticmethod
    def generate_section_html(section_type: str, palette: Dict) -> str:
        """Generate HTML for specific section"""
        sections = {
            'hero': f'''        <section class="hero">
            <div class="hero-content">
                <h1>Welcome to Your Site</h1>
                <p>Create amazing web experiences</p>
                <button class="btn btn-primary">Get Started</button>
            </div>
        </section>''',

            'features': f'''        <section class="features">
            <h2>Features</h2>
            <div class="features-grid">
                <div class="feature-card">
                    <h3>Fast</h3>
                    <p>Lightning-quick performance</p>
                </div>
                <div class="feature-card">
                    <h3>Secure</h3>
                    <p>Enterprise-grade security</p>
                </div>
                <div class="feature-card">
                    <h3>Scalable</h3>
                    <p>Grow without limits</p>
                </div>
            </div>
        </section>''',

            'about': f'''        <section class="about">
            <div class="about-content">
                <h2>About Us</h2>
                <p>We're passionate about creating beautiful, functional websites that help your business grow.</p>
            </div>
        </section>''',

            'portfolio': f'''        <section class="portfolio">
            <h2>Our Work</h2>
            <div class="portfolio-grid">
                <div class="portfolio-item"></div>
                <div class="portfolio-item"></div>
                <div class="portfolio-item"></div>
            </div>
        </section>''',
```

```python
[ ]
class HTMLPageBuilder:
    """Build styled HTML pages from descriptions"""

    @staticmethod
    def extract_sections(description: str) -> List[str]:
        """Extract sections from description"""
        sections = []

        section_keywords = {
            'hero': ['hero', 'banner', 'landing'],
            'features': ['features', 'services', 'benefits'],
            'about': ['about', 'team', 'company'],
            'portfolio': ['portfolio', 'gallery', 'work', 'projects'],
            'testimonials': ['testimonial', 'reviews', 'feedback'],
            'pricing': ['pricing', 'plans', 'cost'],
            'contact': ['contact', 'form', 'reach'],
            'footer': ['footer', 'bottom']
        }

        desc_lower = description.lower()
```

```
        </section>''',

            'testimonials': f'''        <section class="testimonials">
            <h2>What People Say</h2>
            <div class="testimonials-grid">
                <div class="testimonial">
                    <p>"Amazing service!"</p>
                    <span>- Client Name</span>
                </div>
                <div class="testimonial">
                    <p>"Highly recommended!"</p>
                    <span>- Happy Customer</span>
                </div>
            </div>
        </section>''',

            'pricing': f'''        <section class="pricing">
            <h2>Pricing Plans</h2>
            <div class="pricing-grid">
                <div class="pricing-card">
                    <h3>Starter</h3>
                    <p class="price">$29</p>
                    <ul><li>Feature 1</li><li>Feature 2</li></ul>
                </div>
                <div class="pricing-card featured">
                    <h3>Pro</h3>
                    <p class="price">$79</p>
                    <ul><li>All Starter</li><li>Feature 3</li></ul>
                </div>
            </div>
        </section>''',

            'contact': f'''        <section class="contact">
            <h2>Get in Touch</h2>
            <form class="contact-form">
                <input type="text" placeholder="Your Name" required>
                <input type="email" placeholder="Your Email" required>
                <textarea placeholder="Your Message" rows="5" required></textarea>
                <button type="submit" class="btn btn-primary">Send Message</button>
            </form>
        </section>''',

            'footer': f'''        <footer class="footer">
            <div class="footer-content">
                <p>&copy; 2024 Your Company. All rights reserved.</p>
                <div class="footer-links">
                    <a href="#privacy">Privacy</a>
                    <a href="#terms">Terms</a>
                    <a href="#contact">Contact</a>
                </div>
            </div>
        </footer>'''
        }

        return sections.get(section_type, '')
```

Activity 2.3: CSS Generation Engine

The CSS generator creates comprehensive styling including:

- CSS variables for dynamic color management
- Gradient backgrounds and modern animations
- Responsive grid layouts with Flexbox and CSS Grid

- Smooth transitions and hover effects
- Mobile-first media queries for tablets and phones
- Sticky header with backdrop blur effect
- Card-based components with depth and shadows

```
        return sections.get(section_type,    )

    @staticmethod
    def generate_css(palette: Dict) -> str:
        """Generate comprehensive CSS"""
        css = f'''    <style>
        * {{
            margin: 0;
            padding: 0;
            box-sizing: border-box;
        }}

        :root {{
            --primary: {palette['primary']};
            --secondary: {palette['secondary']};
            --accent: {palette['accent']};
            --background: {palette['background']};
            --surface: {palette['surface']};
            --text-primary: {palette['text_primary']};
            --text-secondary: {palette['text_secondary']};
            --border: {palette['border']};
        }}

        body {{
            font-family: 'Segoe UI', Tahoma, Geneva, Verdana, sans-serif;
```

MILESTONE 3: Gradio Interface Development

Activity 3.1: Main Interface with Multiple Tabs

The Gradio interface includes 5 tabs:

```
body {{
    font-family: 'Segoe UI', Tahoma, Geneva, Verdana, sans-serif;
    background: linear-gradient(135deg, var(--background) 0%, var(--surface) 100%);
    color: var(--text-primary);
    line-height: 1.6;
    min-height: 100vh;
}}

header {{
    background: rgba(15, 23, 42, 0.95);
    backdrop-filter: blur(10px);
    padding: 1.5rem 2rem;
    position: sticky;
    top: 0;
    z-index: 100;
    border-bottom: 1px solid var(--border);
}}

header nav {{
    display: flex;
    justify-content: space-between;
    align-items: center;
    max-width: 1200px;
    margin: 0 auto;
}}

header h1 {{
    font-size: 1.8rem;
    background: linear-gradient(135deg, var(--primary), var(--accent));
    -webkit-background-clip: text;
    -webkit-text-fill-color: transparent;
    background-clip: text;
}}

header nav a {{
    color: var(--text-secondary);
    text-decoration: none;
    margin: 0 1.5rem;
    transition: color 0.3s ease;
}}

header nav a:hover {{
    color: var(--primary);
}}

main {{
    max-width: 1200px;
    margin: 0 auto;
    padding: 0 2rem;
}}

section {{
    padding: 4rem 0;
    animation: fadeIn 0.6s ease-in;
}}

@keyframes fadeIn {{
    from {{
        opacity: 0;
        transform: translateY(20px);
    }}
    to {{
        opacity: 1;
        transform: translateY(0);
    }}
}}

h1, h2, h3 {{
    color: var(--text-primary);
    margin-bottom: 1rem;
}}
```

```
h1 {{
    font-size: 3rem;
    font-weight: 700;
}}

h2 {{
    font-size: 2rem;
    margin-bottom: 2rem;
    text-align: center;
}}

h3 {{
    font-size: 1.3rem;
}}

.hero {{
    background: linear-gradient(135deg, var(--primary) 0%, var(--secondary) 100%);
    color: white;
    padding: 6rem 0;
    text-align: center;
    min-height: 500px;
    display: flex;
    align-items: center;
    justify-content: center;
}}

.hero h1 {{
    color: white;
    font-size: 4rem;
    margin-bottom: 1rem;
    animation: slideDown 0.8s ease;
}}

.hero p {{
    font-size: 1.5rem;
    margin-bottom: 2rem;
    opacity: 0.95;
    animation: slideUp 0.8s ease;
}}

@keyframes slideDown {{
    from {{
        opacity: 0;
        transform: translateY(-30px);
    }}
    to {{
        opacity: 1;
        transform: translateY(0);
    }}
}}

@keyframes slideUp {{
    from {{
        opacity: 0;
        transform: translateY(30px);
    }}
    to {{
        opacity: 1;
        transform: translateY(0);
    }}
}}
```

```
.btn {{
    padding: 0.8rem 2rem;
    border: none;
    border-radius: 8px;
    font-size: 1rem;
    cursor: pointer;
    transition: all 0.3s ease;
    font-weight: 600;
}}

.btn-primary {{
    background: var(--primary);
    color: white;
    box-shadow: 0 4px 15px rgba(0,0,0,0.3);
}}

.btn-primary:hover {{
    transform: translateY(-2px);
    box-shadow: 0 6px 20px rgba(0,0,0,0.4);
    background: var(--accent);
}}

.btn-secondary {{
    background: var(--secondary);
    color: white;
}}

.btn-secondary:hover {{
    background: var(--primary);
}}

.features-grid {{
    display: grid;
    grid-templ[illegible]lumns: repeat(auto-fit, minmax(250px, 1fr));
    gap: 2rem;
}}

.feature-card {{
    background: rgba(30, 41, 59, 0.8);
    padding: 2rem;
    border-radius: 12px;
    border: 1px solid var(--border);
    transition: all 0.3s ease;
    backdrop-filter: blur(10px);
}}

.feature-card:hover {{
    transform: translateY(-8px);
    border-color: var(--primary);
    box-shadow: 0 20px 40px rgba(0,0,0,0.3);
}}

.feature-card h3 {{
    color: var(--primary);
    margin-bottom: 0.5rem;
}}

.feature-card p {{
    color: var(--text-secondary);
}}
```

```
.portfolio-grid {{
    display: grid;
    grid-template-columns: repeat(auto-fit, minmax(280px, 1fr));
    gap: 2rem;
}}

.portfolio-item {{
    aspect-ratio: 1;
    background: linear-gradient(135deg, var(--primary), var(--secondary));
    border-radius: 12px;
    cursor: pointer;
    transition: all 0.3s ease;
    border: 1px solid var(--border);
}}

.portfolio-item:hover {{
    transform: scale(1.05);
    box-shadow: 0 20px 40px rgba(0,0,0,0.3);
}}

.about {{
    text-align: center;
}}

.about-content {{
    max-width: 800px;
    margin: 0 auto;
}}

.about p {{
    font-size: 1.1rem;
    color: var(--text-secondary);
}}

.testimonials-grid {{
    display: grid;
    grid-template-columns: repeat(auto-fit, minmax(300px, 1fr));
    gap: 2rem;
}}

.testimonial {{
    background: rgba(30, 41, 59, 0.8);
    padding: 2rem;
    border-left: 4px solid var(--primary);
    border-radius: 8px;
}}

.testimonial p {{
    font-style: italic;
    margin-bottom: 1rem;
    color: var(--text-secondary);
}}

.testimonial span {{
    font-weight: 600;
    color: var(--primary);
}}

.pricing-grid {{
    display: grid;
    grid-template-columns: repeat(auto-fit, minmax(300px, 1fr));
    gap: 2rem;
}}
```

```
.pricing-card {{
    background: rgba(30, 41, 59, 0.8);
    padding: 2rem;
    border-radius: 12px;
    border: 1px solid var(--border);
    text-align: center;
    transition: all 0.3s ease;
}}

.pricing-card.featured {{
    border-color: var(--primary);
    box-shadow: 0 20px 40px rgba(0,0,0,0.3);
    transform: scale(1.05);
}}

.pricing-card .price {{
    font-size: 2.5rem;
    color: var(--primary);
    margin: 1rem 0;
    font-weight: 700;
}}

.pricing-card ul {{
    list-style: none;
    margin: 1.5rem 0;
    text-align: left;
}}

.pricing-card li {{
    padding: 0.5rem 0;
    color: var(--text-secondary);
}}

.contact {{
    max-width: 600px;
    margin: 0 auto;
}}

.contact-form {{
    display: flex;
    flex-direction: column;
    gap: 1.5rem;
}}

.contact-form input,
.contact-form textarea {{
    padding: 1rem;
    border: 1px solid var(--border);
    border-radius: 8px;
    background: rgba(30, 41, 59, 0.8);
    color: var(--text-primary);
    font-family: inherit;
    font-size: 1rem;
}}

.contact-form input:focus,
.contact-form textarea:focus {{
    outline: none;
    border-color: var(--primary);
    box-shadow: 0 0 0 3px rgba(0,0,0,0.3);
}}
```

```
        .footer-content {{
            max-width: 1200px;
            margin: 0 auto;
            padding: 0 2rem;
            display: flex;
            justify-content: space-between;
            align-items: center;
            flex-wrap: wrap;
            gap: 2rem;
        }}

        .footer-links {{
            display: flex;
            gap: 2rem;
        }}

        .footer-links a {{
            color: var(--text-secondary);
            text-decoration: none;
            transition: color 0.3s ease;
        }}

        .footer-links a:hover {{
            color: var(--primary);
        }}

        @media (max-width: 768px) {{
            header nav {{
                flex-direction: column;
                gap: 1rem;
            }}

            header h1 {{
                font-size: 1.4rem;
            }}

            .hero h1 {{
                font-size: 2rem;
            }}

            .hero p {{
                font-size: 1.1rem;
            }}

            h2 {{
                font-size: 1.5rem;
            }}

            main {{
                padding: 0 1rem;
            }}

            section {{
                padding: 2rem 0;
            }}

            .footer-content {{
                flex-direction: column;
                text-align: center;
            }}
        }}
    </style>'''
        return css
```

```python
        # Generate CSS
        css = HTMLPageBuilder.generate_css(palette)

        # Build complete HTML
        html = f'''<!DOCTYPE html>
<html lang="en">
<head>
{meta_tags}
{css}
</head>
<body>
    <header>
        <nav>
            <h1>{title}</h1>
            <div>
                <a href="#features">Features</a>
                <a href="#about">About</a>
                <a href="#contact">Contact</a>
            </div>
        </nav>
    </header>

    <main>
{sections_html}
    </main>
</body>
</html>'''

        return html, palette

generation_history = []

def generate_html_page(title: str, description: str, style: str = "modern") -> Tuple[str, str, str]:
    """Generate HTML page with styling"""
    if not title.strip() or not description.strip():
        return "", "❌ Please provide both title and description!", ""

    try:
        html_output, palette = HTMLGenerationEngine.generate_page(title, description, style)

        color_info = f"""
**Color Palette:**
- Primary: {palette['primary']}
- Secondary: {palette['secondary']}
- Accent: {palette['accent']}
- Background: {palette['background']}
"""

        generation_history.append({
            'title': title,
            'description': description,
            'html': html_output,
            'palette': palette,
            'timestamp': datetime.now().isoformat()
        })
```

```python
class HTMLGenerationEngine:
    """Main engine for HTML page generation"""

    @staticmethod
    def generate_page(title: str, description: str, style: str = "modern") -> Tuple[str, str]:
        """Generate complete HTML page"""

        # Generate color palette
        palette = ColorPaletteGenerator.generate_palette(style)

        # Extract sections
        sections = HTMLPageBuilder.extract_sections(description)

        # Generate meta tags
        meta_tags = HTMLPageBuilder.generate_meta_tags(title, description)

        # Generate sections
        sections_html = '\n\n'.join([
            HTMLPageBuilder.generate_section_html(section, palette)
            for section in sections
            if HTMLPageBuilder.generate_section_html(section, palette)
        ])
```

```
        return html_output, "✅ HTML page generated successfully!", color_info
    except Exception as e:
        return "", f"❌ Error: {str(e)}", ""

def get_color_suggestions(style: str) -> str:
    """Get color suggestions based on style"""
    palette = ColorPaletteGenerator.generate_palette(style)

    palette_text = f"""
**Palette for "{style}" style:**
- Primary: {palette['primary']}
- Secondary: {palette['secondary']}
- Accent: {palette['accent']}
- Background: {palette['background']}
- Text: {palette['text_primary']}
"""
    return palette_text

print("🚀 Loading HTML Quick-Styler Advanced...")

with gr.Blocks(title="HTML Quick-Styler - Advanced Edition") as demo:

    gr.Markdown("""
    # 🎨 HTML Quick-Styler Advanced
    ## Professional HTML Page Generator with Auto-Styling

    **Generate production-ready HTML pages in seconds**
    """)

    with gr.Tabs():
```

Tab 1: Generate Page

```
            )

            color_style = gr.Dropdown(
                ["modern", "vibrant", "ocean", "forest", "sunset", "luxury", "creative", "minimal"],
                value="modern",
                label="🎨 Color Style"
            )

            generate_btn = gr.Button("✨ Generate HTML", size="lg", variant="primary")

        with gr.Column(scale=1):
            gr.Markdown("### 📊 Output")

            html_output = gr.Code(
                language="html",
                label="Generated HTML/CSS",
                lines=20
            )

            status_msg = gr.Markdown()
            color_info = gr.Markdown()

    generate_btn.click(
        generate_html_page,
        inputs=[page_title, page_description, color_style],
        outputs=[html_output, status_msg, color_info]
    )
```

Tab 2: Live Preview

```
            outputs=[html_output, status_msg, color_info]
        )

    # ============= TAB 2: Preview =============
    with gr.TabItem("👁️ Live Preview"):
        gr.Markdown("### 🔍 Preview Generated Page")

        preview_button = gr.Button("🔄 Load Preview", size="lg", variant="primary")
        preview_output = gr.HTML(value="<p style='text-align:center; color:#fff; padding:40px;'>No preview loaded. Generate a page first.</p>")

        def show_preview(html_text):
            if not html_text.strip():
                return "<p style='text-align:center; color:#fff; padding:40px;'>No HTML provided</p>"
            return html_text

        preview_button.click(
            show_preview,
            inputs=[html_output],
            outputs=[preview_output]
        )

    # ============= TAB 3: Color Palettes =============
    with gr.TabItem("🎨 Color Palettes"):
        gr.Markdown("### 🌈 Explore Color Schemes")

        with gr.Row()
```

Tab 3: Color Palettes

```
            outputs=[preview_output]
        )

    # ============= TAB 3: Color Palettes =============
    with gr.TabItem("🎨 Color Palettes"):
        gr.Markdown("### 🌈 Explore Color Schemes")

        with gr.Row():
            style_selector = gr.Radio(
                ["modern", "vibrant", "ocean", "forest", "sunset", "luxury", "creative", "minimal"],
                value="modern",
                label="Choose Style"
            )

            palette_btn = gr.Button("🎨 Get Palette", size="lg", variant="primary")

        palette_display = gr.Markdown()

        palette_btn.click(
            get_color_suggestions,
            inputs=[style_selector],
            outputs=[palette_display]
        )
```

Tab 4: Templates

```python
# ============= TAB 4: Templates =============
with gr.TabItem("📋 Templates"):
    gr.Markdown("### 📝 Quick Templates")

    with gr.Row():
        with gr.Column():
            gr.Markdown("""
            #### Portfolio Website
            Hero, Portfolio Gallery, About, Contact, Footer
            """)
            portfolio_btn = gr.Button("Use Template", size="sm")

            def load_portfolio():
                return ("Portfolio Website",
                        "Hero section with welcome banner, portfolio gallery with 6 items, about section, testimonials, contact form, foot
                        "modern")

            portfolio_btn.click(load_portfolio, outputs=[page_title, page_description, color_style])

        with gr.Column():
            gr.Markdown("""
            #### SaaS Landing Page
            Hero, Features, Pricing, Testimonials, CTA, Footer
            """)
            saas_btn = gr.Button("Use Template", size="sm")
```

```python
            saas_btn = gr.Button("Use Template", size="sm")

            def load_saas():
                return ("SaaS Product",
                        "Hero banner, features showcase with 3 items, pricing plans comparison, testimonials, call-to-action button, footer",
                        "vibrant")

            saas_btn.click(load_saas, outputs=[page_title, page_description, color_style])

        with gr.Column():
            gr.Markdown("""
            #### Company Website
            Header, About, Services, Team, Contact, Footer
            """)
            company_btn = gr.Button("Use Template", size="sm")

            def load_company():
                return ("Company",
                        "Header with navigation, hero section, about company, services list, team members, contact form, footer",
                        "ocean")

            company_btn.click(load_company, outputs=[page_title, page_description, color_style])
```

Tab 5: Guide

```python
# ============= TAB 5: Guide =============
with gr.TabItem("📚 Guide"):
    gr.Markdown("""
    # 📖 HTML Quick-Styler Guide

    ## How It Works

    ### 1. Enter Details
    - **Title:** Page name
    - **Description:** Sections you want
    - **Color Style:** Pick a theme

    ### 2. Automatic Detection
    The tool recognizes these keywords:
    - `hero` / `banner` → Hero Section
    - `features` / `services` → Features Grid
    - `about` / `team` → About Section
    - `portfolio` / `gallery` → Portfolio
    - `testimonial` / `reviews` → Testimonials
    - `pricing` / `plans` → Pricing Plans
    - `contact` / `form` → Contact Form
    - `footer` → Footer
```

MILESTONE 4: Deployment

Activity 4.1: Google Colab Deployment

Launch the application in Google Colab:

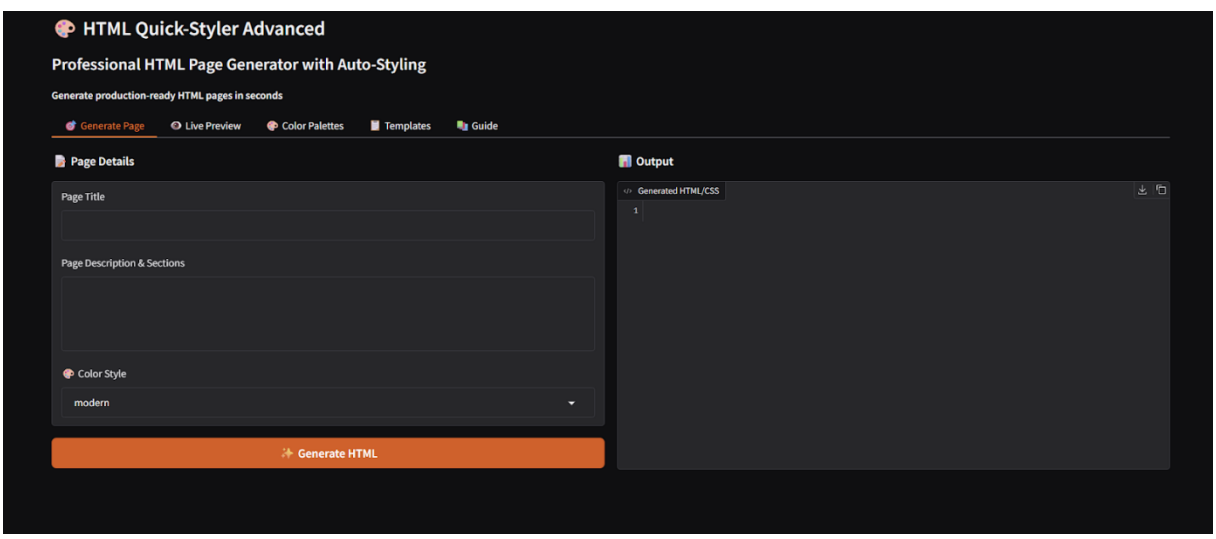
```
print("\n✅ HTML Quick-Styler Advanced Ready!\n")
demo.launch(share=True, show_error=True, show_api=False)
```

```
🚀 Loading HTML Quick-Styler Advanced...
ut actions
L Quick-Styler Advanced Ready!

/tmp/ipython-input-1159347522.py:956: DeprecationWarning: The 'show_api' parameter in launch() will be removed
  demo.launch(share=True, show_error=True, show_api=False)
Colab notebook detected. To show errors in colab notebook, set debug=True in launch()
* Running on public URL: https://87205c7f5a55c5adc8.gradio.live
```

The application will be accessible via:

- Public URL: https://3a78a5dd569725781f.gradio.live



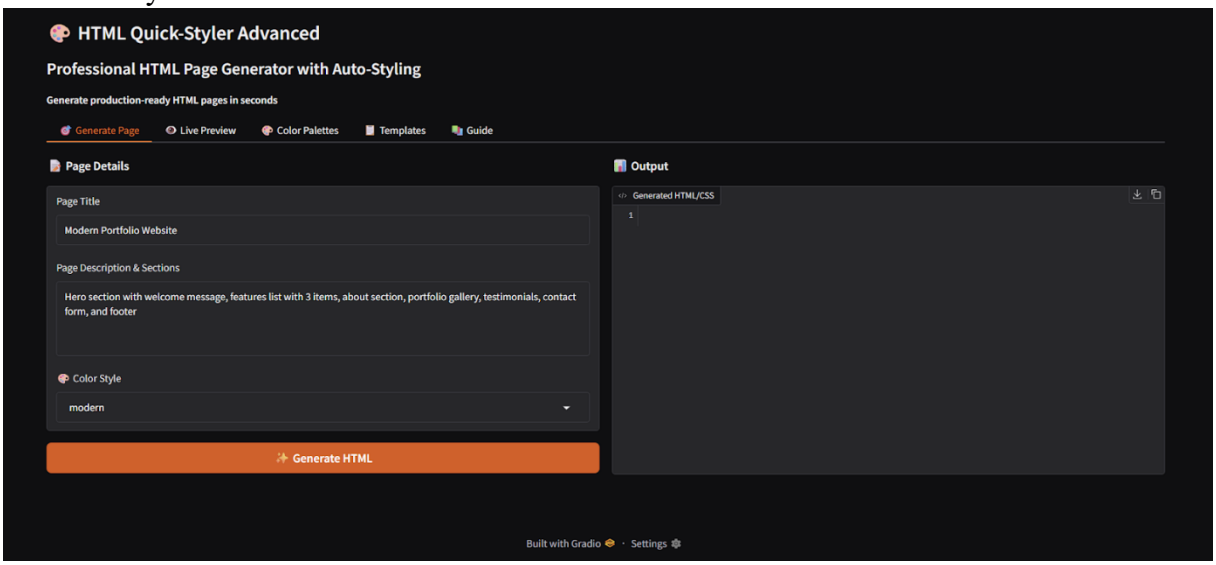
MILESTONE 5: Testing & Validation

Activity 5.1: Functional Testing

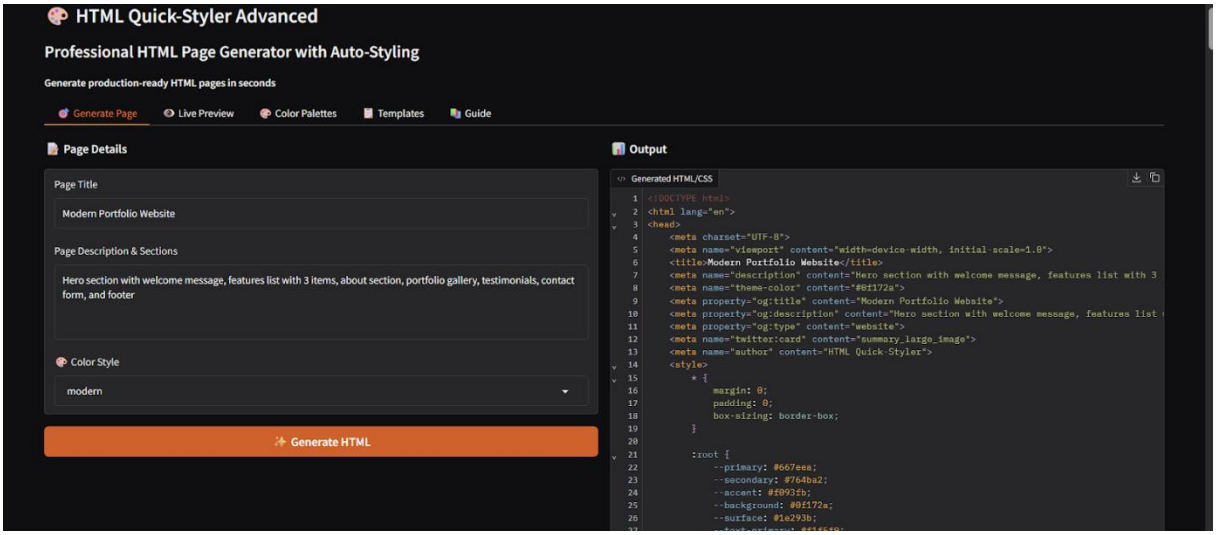
Test Case 1: Portfolio Website Generation

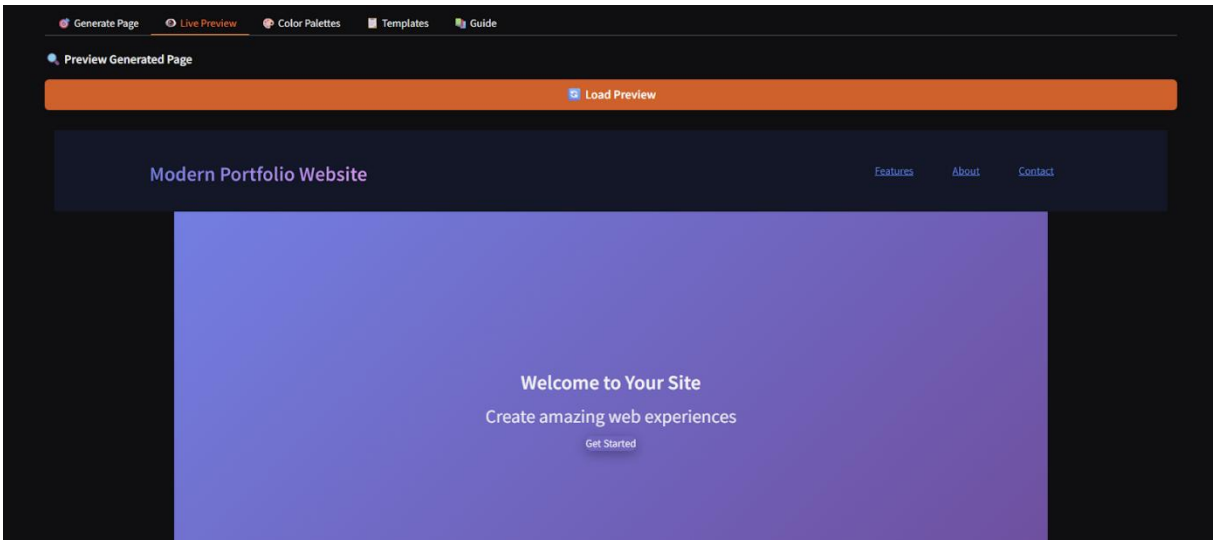
Input:

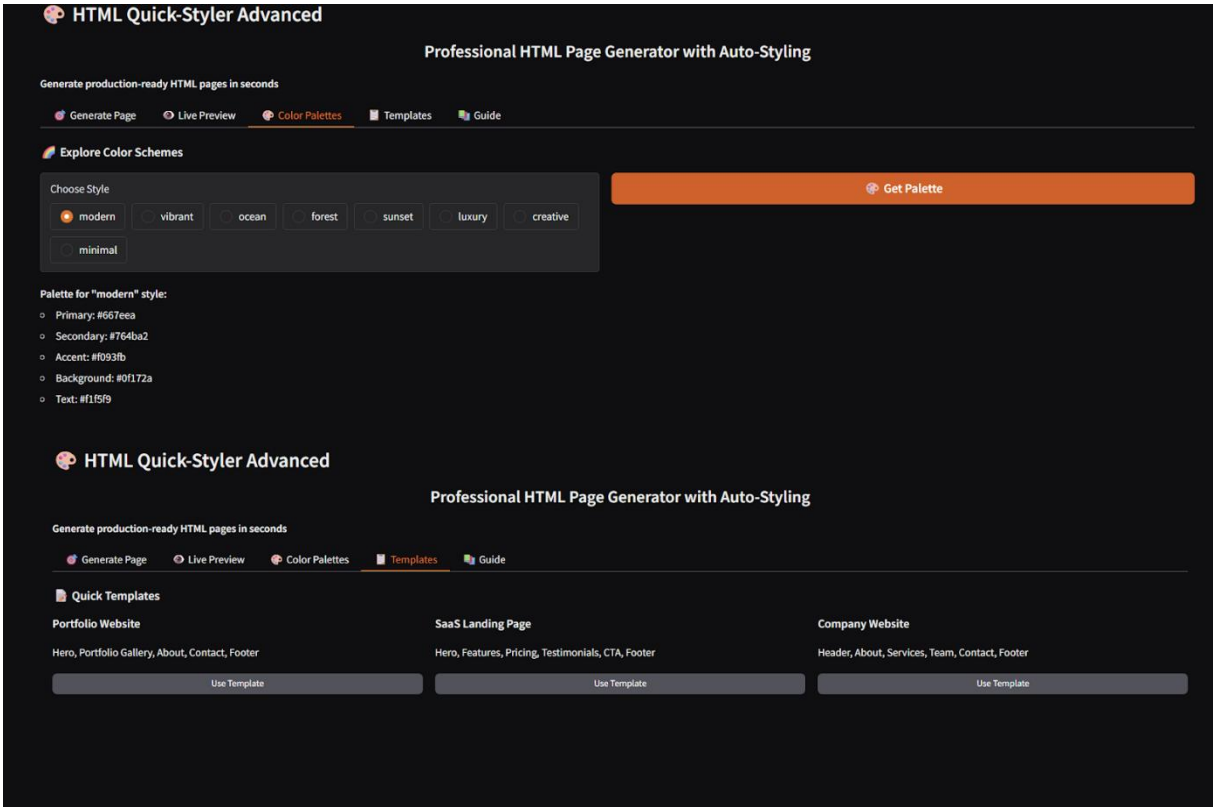
- Title: Modern Portfolio Website
- Description: Hero section with welcome message, features list with 3 items, about section, portfolio gallery, testimonials, contact form, and footer
- Color Style: modern



Expected Output:




Generate Page
Live Preview
Color Palettes
Templates
Guide
Preview Generated Page
Load Preview
Modern Portfolio Website
Features
About
Contact
Welcome to Your Site
Create amazing web experiences
Get Started


HTML Quick-Styler Advanced
Professional HTML Page Generator with Auto-Styling
Generate production-ready HTML pages in seconds
Generate Page
Live Preview
Color Palettes
Templates
Guide
Explore Color Schemes
Choose Style
modern
vibrant
ocean
forest
sunset
luxury
creative
minimal
Get Palette
Palette for "modern" style:
Primary: #667eea
Secondary: #764ba2
Accent: #f093fb
Background: #0f172a
Text: #f1f5f9
HTML Quick-Styler Advanced
Professional HTML Page Generator with Auto-Styling
Generate production-ready HTML pages in seconds
Generate Page
Live Preview
Color Palettes
Templates
Guide
Quick Templates
Portfolio Website
Hero, Portfolio Gallery, About, Contact, Footer
Use Template
SaaS Landing Page
Hero, Features, Pricing, Testimonials, CTA, Footer
Use Template
Company Website
Header, About, Services, Team, Contact, Footer
Use Template

HTML Quick-Styler Advanced

Professional HTML Page Generator with Auto-Styling

Generate production-ready HTML pages in seconds

Generate Page | Live Preview | Color Palettes | Templates | Guide

HTML Quick-Styler Guide

How It Works

1. Enter Details

- **Title:** Page name
- **Description:** Sections you want
- **Color Style:** Pick a theme

2. Automatic Detection

The tool recognizes these keywords:

- `hero` / `banner` → Hero Section
- `features` / `services` → Features Grid
- `about` / `team` → About Section
- `portfolio` / `gallery` → Portfolio
- `testimonial` / `reviews` → Testimonials
- `pricing` / `plans` → Pricing Plans
- `contact` / `form` → Contact Form
- `footer` → Footer

3. Generate & Customize

CONCLUSION

Conclusion

HTML Quick-Styler successfully delivers an intelligent web page generation assistant that combines intuitive design templates with IBM Granite 3.3 2B Instruct model capabilities. By leveraging Gradio's powerful interface framework and Google Colab's cloud infrastructure, the system provides fast, professional, and customizable HTML/CSS generation accessible to designers, developers, and non-technical users alike.

The platform demonstrates the viability of AI-assisted web development, making professional page creation accessible while maintaining high design standards, responsive layouts, and production-ready code quality. With eight distinct color palettes, intelligent section detection, and comprehensive template options, HTML Quick-Styler empowers users to create beautiful websites in minutes rather than hours.

Future enhancements could include custom color picker integration, JavaScript interactivity generation, component library expansion, and integration with popular web frameworks. The modular architecture allows for easy extension and customization to meet evolving user needs and design trends.